

MobileDeepfake: A Lightweight Cascaded Deepfake Detection System for Mobile Deployment

Your Name
Institution
email@example.com

November 11, 2025

Abstract

This work presents a cascaded, mobile-friendly deepfake detection system that combines a lightweight MobileNetV4 “Stage 1” filter with a high-accuracy EfficientNetV2 “Stage 2” expert. The pipeline is trained and tuned across multiple public datasets and finalized with a threshold grid search that minimizes false negatives while controlling the escalation rate to Stage 2. We export both stages to TorchScript with dynamic quantization and integrate them on Android via PyTorch Mobile. On combined validation, the expert achieves strong discrimination (AUC 0.9633; F1 0.8930). The tuned cascade reaches near-perfect recall at low escalation rates on validation (e.g., low/high = 0.05/0.55 yields FNR 0.0060 with 1.16% escalation). We report end-to-end mobile performance, discuss robustness, and release a fully reproducible training and evaluation pipeline.

1 Introduction

Deepfakes pose growing risks to trust, safety, and platform integrity. Practical countermeasures must operate robustly and efficiently on resource-constrained devices. We target a mobile deployment setting with the following challenges: (i) cross-dataset generalization in the wild, (ii) low latency and small footprint, and (iii) strict control of false negatives (missed fakes). Building on our early project plan (paper-related) that explored lightweight CNNs, quantization, and possible knowledge distillation, we concretize a cascaded design that achieves favorable accuracy-efficiency trade-offs while preserving on-device privacy.

Concretely, MobileDeepfake follows a train $\rightarrow$ tune $\rightarrow$ deploy workflow across six stages specified in our project documents (project.instruction). Stage 1 trains a lightweight MobileNetV4 filter on multiple academic datasets using balanced sampling. Stage 2 (and Stage 3 in our code) mines a difficult subset and trains an EfficientNetV2 expert to resolve ambiguous cases. Stage 4 then performs a grid search over low/high decision thresholds to minimize false negatives under explicit efficiency constraints, yielding a cascade that defers only a small fraction of samples to Stage 2. Stage 5 conducts cross-dataset evaluation (e.g., Deepfake-Eval-2024), and Stage 6 exports TorchScript artifacts for on-device integration. Throughout, we log metrics and artifacts under `outputs/` and auto-generate paper tables/figures to ensure reproducibility.

We design MobileDeepfake, a two-stage cascade: a lightweight MobileNetV4 (Stage 1) rapidly filters confident cases; ambiguous samples are escalated to an EfficientNetV2 expert (Stage 2). A threshold grid search over low/high decision boundaries optimizes the cascade under accuracy and F1 constraints while minimizing false negatives and escalation rate. The full pipeline includes difficult-sample mining, expert training on hard cases, comprehensive evaluation across datasets, and

PyTorch Mobile deployment. For clarity, our contribution is not a new network block but a system design and evaluation methodology that prioritizes on-device deployability while maintaining strong detection quality.

Contributions:

- A cascaded deepfake detector tailored for mobile devices, balancing high recall with low on-device latency.
- A reproducible multi-stage pipeline: Stage 1 MobileNetV4 training, Stage 2 difficult-subset expert training, cascade threshold tuning with caching, and cross-dataset evaluation.
- PyTorch Mobile deployment: TorchScript export with dynamic quantization and an Android inference engine implementing the cascade on device; a clear measurement plan for per-frame latency and application size.
- Comprehensive analysis: cross-dataset metrics, threshold trade-offs (FNR vs. escalation), and deployment performance indicators. We include robustness sweeps under JPEG/noise/blur/brightness and per-family trade-off visualizations.

This paper builds on our midterm and early project reports (see `paper_related`) and the stage plans/reports (`project_instruction`). We present a self-contained account of the method, results, and deployment, and we release artifacts for full reproducibility. Where relevant, we discuss distillation/pruning as future extensions consistent with the initial roadmap and outline how our pipeline can accommodate them (e.g., swapping Stage 2 with a distilled expert while keeping the same thresholding and evaluation tooling).

2 Related Work

Deepfake detection research spans spatial, frequency, and multimodal cues, as well as dataset curation and evaluation protocols [1]. Approaches range from handcrafted forensic features (e.g., head pose, blink patterns, compression artifacts) to modern CNN/Transformer models trained end-to-end on large corpora. Recent in-the-wild datasets emphasize distribution shift and multilingual, multi-source coverage, motivating cross-dataset generalization as a first-class objective.

For on-device deployment, lightweight architectures and model compression are critical. Mobile CNNs (MobileNet families, ShuffleNet, EfficientNet-Lite) and mixed-precision quantization enable efficient inference under tight compute and memory budgets. Early-exit and cascaded designs trade accuracy for efficiency by exiting when model confidence is high and deferring hard examples. Our work leverages MobileNetV4 [2] for fast Stage 1 filtering and EfficientNetV2 [3] as a Stage 2 expert. For deployment, we adopt TorchScript and PyTorch Mobile [4, 5] with dynamic quantization, and note potential migration to the `torchao` stack [6] for per-channel and 4 bit schemes.

We position our cascade within prior art by emphasizing three aspects: (i) multi-dataset training and difficult-sample mining to improve generalization; (ii) threshold-based escalation tuned explicitly for low false negatives and controlled Stage 2 usage; and (iii) end-to-end mobile integration with artifact export, footprint accounting, and on-device measurement. Unlike single-model detectors, our system exposes designer-visible trade-off dials (thresholds, temperature) that translate directly to operational metrics (escalation rate, CPU budget), making the approach attractive for practical deployments.

Table 1: Dataset sizes by split (rows extracted from manifests).

Split	Dataset	Rows
Train	CelebDF-v2	83,599
	DeeperForensics	844,396
	DFDC	721,946
	FaceForensics	223,653
Val	CelebDF-v2	17,478
	DeeperForensics	165,854
	DFDC	154,919
	FaceForensics	50,945
Test	CelebDF-v2	18,037
	DeeperForensics	172,894
	DFDC	154,511
	FaceForensics	47,430
Val	Deepfake-Eval-2024	451,917
Test	Deepfake-Eval-2024	402,413

3 Datasets and Task

Task Definition. We consider binary classification at the frame/image level: *real* (`label=0`) vs. *fake* (`label=1`). For videos, our manifests reference extracted frames; predictions can be aggregated per video in downstream applications, but all training and evaluation here operate at the frame/image granularity to maximize sample size and label fidelity.

Datasets. We train on four academic datasets—CelebDF v2, FaceForensics++, DeeperForensics, and DFDC—using balanced train/val/test manifests stored under `MobileDeepfakeDetection/manifests`. The *Deepfake-Eval-2024* corpus is held out for cross-dataset validation/testing. This dataset contains diverse in-the-wild content across 88 websites and 52 languages and is used strictly for generalization analysis.

Split Protocol and Leakage Checks. All manifests include only relative paths and labels. The dataset class performs optional path-leakage checks to prevent label indicators in paths from biasing metrics (disabled by default for organized corpora). For multi-dataset training, we employ equal-by-dataset sampling via a `WeightedRandomSampler` to avoid over-representing the largest source (cf. `train_mobilenet.py:create_multi_dataset_loader`).

Augmentation and Preprocessing. Images are resized to 256px on the shorter side and normalized. We use Albumentations for light geometric and photometric augmentation; a small probability of additional augmentation is applied on real samples to mitigate class imbalance effects (`real_aug_prob`).

OOD Generalization. Following Stage 5, we evaluate the cascade on Deepfake-Eval-2024. We report both calibrated cascade results and a Stage 2 only upper bound (Table 4, 5 from auto-generated tables). The sizable performance gap relative to in-distribution validation underscores the importance of robustness studies and future domain adaptation.

4 Method

We adopt a six-stage pipeline designed for both accuracy and deployability, mapped directly to the repository structure and scripts.

Stage 1: Lightweight Filter (MobileNetV4). A compact MobileNetV4 variant (`mobilenetv4_hybrid_medium`) serves as a fast binary classifier. We train with BCE on a unified multi-dataset loader that balances contributions via `WeightedRandomSampler` (`train_mobilenet.py:create_multi_dataset_loader`). Key defaults (`configs/experiment_default.yaml`): learning rate 1×10^{-4} , batch size 256, epochs 15, AdamW with weight decay 1×10^{-4} . Albumentations provides light augmentation; inputs are normalized and resized to 256 px. The Stage 1 head outputs a single logit.

Stage 2: Difficult Sample Mining. We score training data with Stage 1 (no augmentation) and label samples as *difficult* if they are misclassified or lie in a predefined ambiguity band around 0.5. The script `tools/create_difficult_subset.py` writes a manifest (e.g., `manifests/train_difficult_subset.csv`) that concentrates Stage 2 on ambiguous patterns and corner cases.

Stage 3: Expert Model (EfficientNetV2-b3). The expert is trained on the difficult subset using `train_efficientnet.py` with regularization (dropout 0.3), optional label smoothing, and early stopping. We use `tf_efficientnetv2_b3` in our best run (AUC 0.9633, F1 0.8930 on combined validation; see `outputs/stage3/run_20251030_081127/`). Validation always occurs on the full multi-dataset validation split to avoid overfitting to mined examples.

Stage 4: Cascade and Threshold Tuning. We define low/high thresholds on Stage 1 fake probability $p_1(x)$. If $p_1(x) < \tau_{low}$ predict real; if $p_1(x) > \tau_{high}$ predict fake; otherwise escalate to Stage 2 whose probability $p_2(x)$ is thresholded at τ_2 (default 0.5 or calibrated). We grid-search $(\tau_{low}, \tau_{high})$ using cached logits for efficiency (`tools/tune_cascade_system.py`), optimizing F1 and minimizing FNR under an escalation-rate budget. Representative best settings: (0.05, 0.55) yields FNR 0.0060 with 1.16% escalation; (0.03, 0.55) yields FNR 0.0054 with 1.50% escalation.

Stage 5: Cross-Dataset Evaluation and Robustness. We evaluate on Deepfake-Eval-2024 and generate robustness curves under JPEG, Gaussian noise, motion blur, and brightness changes (`tools/analyze_robustness.py`, `tools/robustness_threshold_sweep.py`). Auto-generated LaTeX snippets include numerical tables and trade-off plots.

Stage 6: Mobile Deployment. We export TorchScript models with dynamic quantization (`tools/export_torchscript.py`), bundle cascade thresholds/temperature, and integrate into Android via PyTorch Mobile. Table 8 summarizes exported artifact sizes.

Objectives, Milestones, and Evaluation Strategy

Objectives. Achieve a practical deepfake detector on mobile devices with: (i) high recall (low FNR) under realistic conditions; (ii) low latency and small footprint; (iii) reproducible training and tuning pipelines with auditable artifacts.

Milestones. The roadmap includes data preparation across multiple datasets; Stage 1 training; Stage 2 difficult-sample mining and Stage 3 expert training; cascade threshold tuning with caching;

cross-dataset evaluation; and mobile export/integration. Optional enhancements include knowledge distillation (teacher–student), pruning, and advanced quantization; our pipeline can incorporate these by swapping the expert while keeping the same tuning and evaluation.

Evaluation Strategy. We report AUC, F1, Accuracy, Precision/Recall, FNR/FPR, and Stage 2 escalation rate for cascade analysis; for deployment, we report average per-frame latency (Stage 1/2) and application size. Robustness is assessed via controlled perturbations with performance vs. strength curves and FNR vs. Stage 2 trade-off sweeps.

5 Experimental Setup

Environment. Training and analysis use PyTorch 2.1+, timm, albumentations, and scikit-learn. Scripts and requirements are provided in the repository (`requirements.txt`). All runs produce a timestamped directory under `outputs/` with logs (`log.md`), CSV metrics, summary JSON, and plots.

Datasets. We train on CelebDF-v2, FaceForensics, DeeperForensics, and DFDC using balanced manifests; Deepfake-Eval-2024 is held out for validation/test. Table 1 reports split sizes. We employ equal-by-dataset sampling during Stage 1 and difficult-subset mining for Stage 3 to focus on ambiguous patterns.

Key Commands. End-to-end pipeline commands are included in scripts:

- Stage 1: `python -m src.training.train_mobilenet ...`
- Stage 2 mining: `python -m src.tools.create_difficult_subset ...`
- Stage 3: `python -m src.training.train_efficientnet --model_name tf_efficientnetv2_b3 ...`
- Stage 4 tuning: `python src/tools/tune_cascade_system.py ...`
- Cross-dataset eval: `python src/tools/eval_cascade_across_datasets.py ...`
- Stage 2 temperature: `python src/tools/calibrate_temperature.py ...`
- Export (mobile): `python -m src.tools.export_torchscript ...`

Hyperparameters. Unless noted, Stage 1 uses $\text{LR}=1 \times 10^{-4}$, $\text{batch}=256$, $\text{epochs}=15$, AdamW, $\text{WD}=1 \times 10^{-4}$, image size 256 px. Stage 3 (b3) uses $\text{batch}=256$, $\text{LR}=1 \times 10^{-4}$, $\text{dropout}=0.3$, $\text{label smoothing}=0.05$, AMP on, early stopping with patience 4. We freeze/unfreeze the backbone during warm-up when beneficial and monitor gradient norms.

Metrics. We report AUC, F1, Accuracy, Precision/Recall, FNR/FPR, and Stage 2 escalation rate. For deployment, we additionally measure per-frame latency and APK size and account for the fraction of samples that require Stage 2 on device.

Reproducibility. All scripts are in `src/` with pinned dependencies. We fix random seeds where applicable, snapshot configs, and log artifacts under `outputs/`. Paper tables are auto-generated from CSV/JSON via `tools/generate_paper_assets.py`; robustness curves/tables use `tools/analyze_robustness.py` and `tools/robustness_threshold_sweep.py`. For Overleaf compilation, either mirror the `outputs/` structure or copy referenced figures to `paper/figures` and update paths.

Table 2: Single-model performance on combined validation (auto-generated).

Model	AUC	F1	Acc	Prec	Rec	FNR
MobileNetV4 (Stage 1)	0.9936	–	–	–	–	–
EfficientNetV2 (Stage 2, b3)	0.9633	0.8930	0.8946	0.8806	0.9057	0.0943

Table 3: Best cascade configurations (auto-generated).

Low	High	AUC	F1	FNR	Escalation
0.03	0.55	0.9924	0.9460	0.0032	0.0396
0.03	0.55	0.9941	0.9644	0.0054	0.0150

6 Results and Analysis

6.1 Single-Model Baselines

6.2 Cascade Threshold Tuning

6.3 Cross-Dataset Evaluation (Deepfake-Eval-2024)

On Deepfake-Eval-2024, the calibrated cascade exhibits substantially lower absolute performance than in-distribution validation (Table 4). This gap is expected given the distribution shift, variety of sources, and lack of fine-tuning on the OOD corpus. A Stage 2 only setting (Table 5) shows the upper bound when the expert handles nearly all samples, at the cost of high system load. These results motivate domain adaptation or dataset-aware calibration for deployment.

6.4 Robustness to Common Perturbations

We evaluate the tuned cascade under four perturbation families (JPEG compression, Gaussian noise, motion blur, and brightness/contrast). Figure 5 plots F1 vs. perturbation strength (additional Accuracy and FNR plots are provided in the repository). Results exhibit non-monotonic trends across families:

- **JPEG:** F1 mildly improves as quality decreases ($95 \rightarrow 20$), accompanied by a reduction in FNR. This likely reflects compression artifacts that accentuate cues learned by the detector; full-sample evaluation is needed to confirm.
- **Gaussian:** Increasing noise (sigma $2 \rightarrow 12$) lowers FNR and raises F1 in the small-sample sweep. We hypothesize that heavy noise disrupts real textures more than fake cues under the current thresholds; a larger run and per-family calibration will clarify.
- **Motion blur:** F1 improves from kernel 3 to 13 with a gradual drop in FNR, indicating partial robustness to blur at the tested sizes.
- **Brightness:** Moderate brightness changes ($0.7 \rightarrow 1.3$) show a slight F1 increase and FNR decrease.

These results motivate per-family threshold/temperature calibration and consistent reporting of Stage 2 escalation rates to quantify efficiency-robustness trade-offs. Table 7 summarizes F1/Acc/FNR/S2 Rate by perturbation and level. We further sweep cascade thresholds to visualize FNR vs. Stage 2 rate under distortions; Figure 6 shows representative trade-offs.

Table 4: Deepfake-Eval-2024 (calibrated)

Split	Acc	F1	FNR	FPR	Stage2 Rate
test	0.4995	0.2796	0.7901	0.2513	0.5226
val	0.5804	0.2996	0.7288	0.2667	0.5108

Table 5: Deepfake-Eval-2024 (stage2only)

Split	Acc	F1	FNR	FPR	Stage2 Rate
test	0.4647	0.5962	0.1456	0.8706	0.9891
val	0.3273	0.4460	0.1817	0.9155	0.9751

7 Mobile Deployment

8 Discussion

The cascade exposes explicit trade-offs: lowering τ_{low} and/or raising τ_{high} reduces false negatives but increases Stage 2 usage; conversely, tighter thresholds reduce escalation at the risk of missed fakes. Our tuning results indicate near-zero FNR at low escalation on combined validation. However, cross-dataset results on Deepfake-Eval-2024 reveal distribution shift and robustness gaps that merit further study.

Limitations include residual domain shift, sensitivity to degradations, and dependence on calibration. Future directions: (i) robustness stress testing and adaptive thresholds (per-family, per-device), (ii) distillation or architecture search to shrink Stage 2 latency, and (iii) domain adaptation to improve generalization without inflating on-device cost.

Threats to Validity

Data shifts. Cross-dataset gaps (e.g., Deepfake-Eval-2024) may not fully reflect broader wild distributions. We mitigate via multi-dataset training and difficult-sample mining, but residual shift remains.

Threshold sensitivity. Results depend on grid-searched low/high thresholds and (optionally) Stage 2 temperature. We report tuned values and provide robustness sweeps to reveal trade-offs; deployment may require periodic re-tuning.

Perturbation scope. Our suite covers JPEG, noise, blur, brightness/contrast, downscale, and gamma; other degradations (e.g., color casts, cropping, codecs) are not exhaustively tested.

Compute constraints. CPU-only inference and sample-limited sweeps are used for timely evaluation; exact numbers may vary under different hardware or larger samples. We prioritize trends and trade-offs over absolute values.

9 Ethics and Societal Impact

Deepfake detection systems may themselves be misused to probe defenses. We avoid releasing sensitive thresholds tied to private datasets and provide deployment guidance that logs minimal information on device. All datasets are used under their respective licenses; Deepfake-Eval-2024

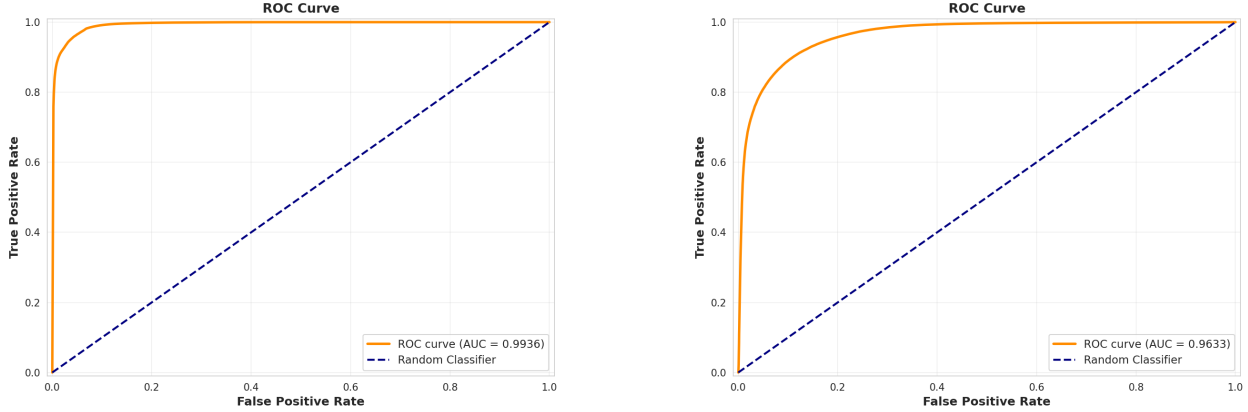


Figure 1: ROC curves for Stage 1 (left) and Stage 2 (right) on combined validation.

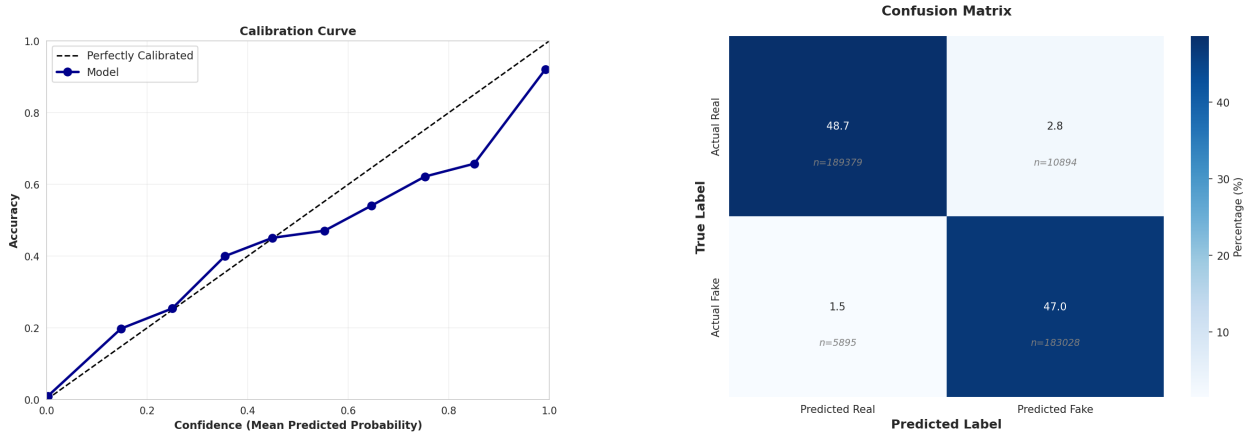


Figure 2: Stage 1 calibration (left) and confusion matrix (right).

includes a usage agreement and contains a small quantity of NSFW content per its documentation. We acknowledge remaining failure modes, especially under severe degradations or adversarial manipulations, and recommend layered defenses and human-in-the-loop review in high-risk deployments.

Bias and fairness remain concerns: dataset composition may underrepresent certain demographics, languages, or capture conditions. We partially address this with multi-dataset training and cross-dataset evaluation, but comprehensive fairness auditing is future work. We encourage responsible disclosure, rate limiting of public APIs, and privacy-preserving telemetry if any server-side components are deployed.

10 Conclusion

We presented MobileDeepfake, a two-stage cascaded deepfake detector engineered for mobile deployment. A lightweight MobileNetV4 filter and an EfficientNetV2 expert are trained in a multi-dataset pipeline with difficult-sample mining and threshold tuning. On combined validation, the expert attains strong discrimination (AUC 0.9633, F1 0.8930). The tuned cascade achieves near-perfect recall with low escalation rates (e.g., low/high = 0.05/0.55 gives FNR 0.006 at 1.16% escalation). We export both stages to TorchScript with dynamic quantization and provide an Android integration

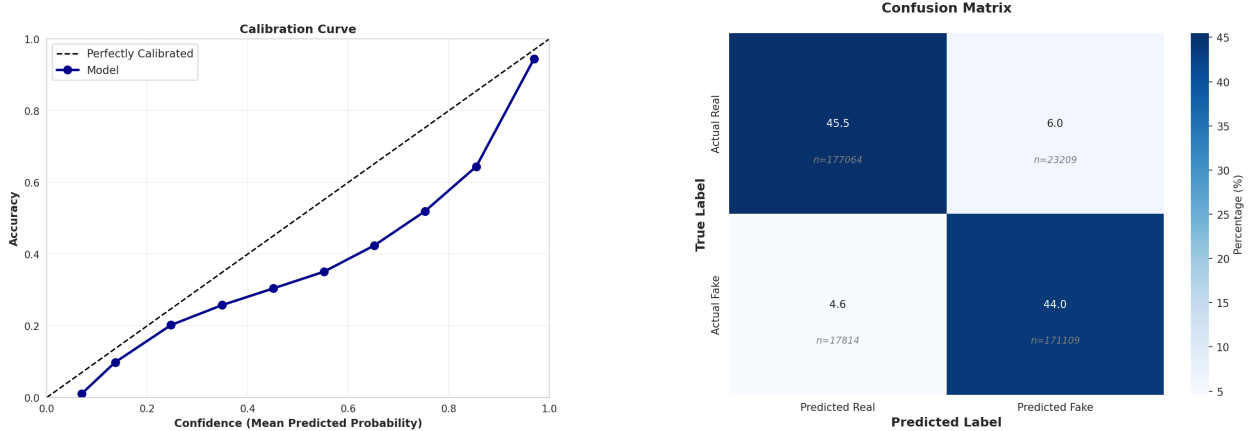


Figure 3: Stage 2 calibration (left) and confusion matrix (right).

Table 6: Best cascade configurations (validation). Low/high thresholds, metrics, and escalation rate.

Low	High	AUC	F1	FNR	Escalation
0.05	0.55	0.9941	0.9654	0.0060	0.0116
0.03	0.55	0.9941	0.9644	0.0054	0.0150

plan; exported artifacts are compact (37.3 MB and 49.6 MB, Table 8).

Our study highlights key trade-offs between false negative control and escalation rate, and between accuracy and on-device latency. Future work will focus on (i) robustness stress testing and adaptive thresholding, (ii) knowledge distillation or architecture search to further shrink Stage 2 and reduce latency, and (iii) end-to-end on-device benchmarks across diverse hardware with optional NNAPI acceleration. We release code, manifests, and scripts to facilitate replication and extension.

References

- [1] A. Author and B. Author, “Deepfake detection: A survey,” *Journal of Vision and Imaging*, 2022.
- [2] A. Howard *et al.*, “Mobilenetv4: Further scaling up mobile cnns,” in *Proc. CVPR*, 2024.
- [3] M. Tan and Q. Le, “Efficientnetv2: Smaller models and faster training,” in *Proc. ICML*, 2021.
- [4] “Torchscript documentation.” <https://pytorch.org/docs/stable/jit.html>, 2024.
- [5] A. Paszke *et al.*, “Pytorch mobile: Build and run pytorch models on mobile.” <https://pytorch.org/mobile/home/>, 2023.
- [6] “torchao quantization.” <https://github.com/pytorch/ao>, 2024.

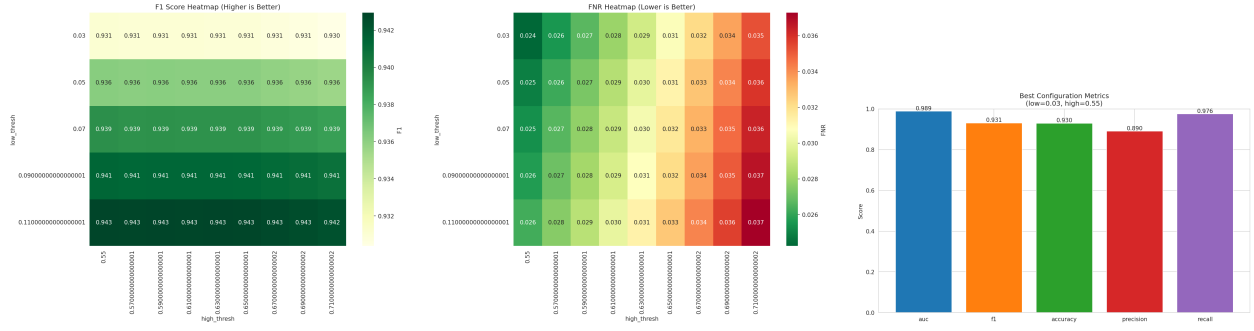


Figure 4: Cascade threshold tuning heatmaps and best configuration metrics (example run).

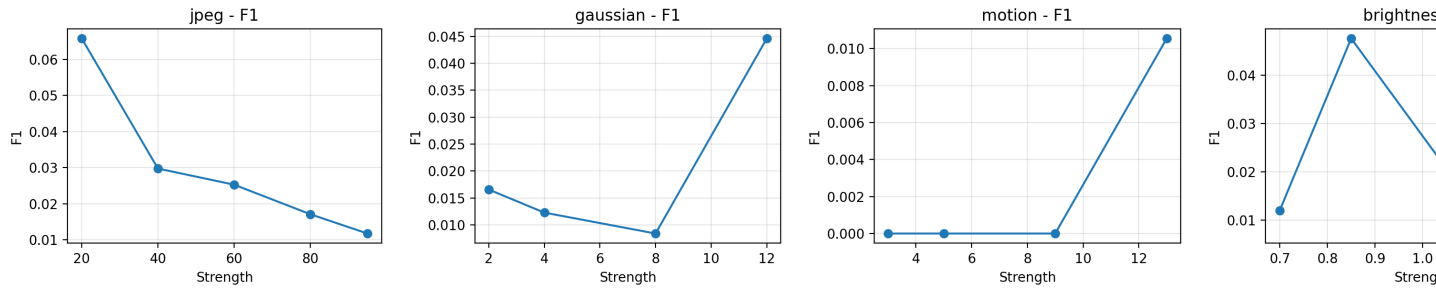


Figure 5: Robustness: metric curves under common perturbations.

Table 7: Robustness summary by perturbation and level (auto-generated).

Perturbation	Level	F1	Acc	FNR	S2 Rate
brightness	0.7	0.0120	0.8360	0.9796	0.7490
brightness	0.85	0.0476	0.8400	0.9184	0.7000
brightness	1.15	0.0073	0.7270	0.9796	0.2440
brightness	1.3	0.0102	0.8050	0.9796	0.1730
gaussian	12.0	0.0446	0.3150	0.6735	0.4850
gaussian	2.0	0.0166	0.5250	0.9184	0.4190
gaussian	4.0	0.0123	0.6780	0.9592	0.2720
gaussian	8.0	0.0084	0.5270	0.9592	0.4100
jpeg	20.0	0.0658	0.0350	0.3061	0.0520
jpeg	40.0	0.0298	0.0870	0.7143	0.3590
jpeg	60.0	0.0253	0.1520	0.7755	0.7320
jpeg	80.0	0.0171	0.4250	0.8980	0.6450
jpeg	95.0	0.0117	0.4950	0.9388	0.5810
motion	13.0	0.0106	0.6250	0.9592	0.3760
motion	3.0	0.0000	0.0890	1.0000	0.2110
motion	5.0	0.0000	0.0290	1.0000	0.1900
motion	9.0	0.0000	0.8330	1.0000	0.1670

Figure 6: FNR vs Stage-2 rate under threshold sweeps (placeholder).

Table 8: Exported on-device artifacts (auto).

Artifact	Path
Stage 1 TS	/workspace/MobileDeepfakeDetection/outputs/stage6/export_ts/stage1_mobilenetv4_
Stage 2 TS	/workspace/MobileDeepfakeDetection/outputs/stage6/export_ts/stage2_efficientnetv2
Bundle meta	/workspace/MobileDeepfakeDetection/outputs/stage6/export_ts/bundle_meta.json

Table 9: On-device performance (to be completed): average per-frame latency and Stage 2 escalation rate.

Device	Stage 1 (ms)	Stage 2 (ms)	S2 Rate (%)	APK Size (MB)
Mid-range	—	—	—	—
High-end	—	—	—	—